

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №5

По дисциплине: «Модели решения задач в И С»

Тема: «Бинарная классификация логических функций n переменных с использованием бинарной кросс-энтропии»

Выполнил:

Студент 3 курса

Группы ИИ-26

Шоева Е.Т.

Проверила:

Андренко К.В.

Цель работы: Реализовать однослойный персептрон с сигмоидной функцией активации и функцией потерь Binary Cross-Entropy (BCE) для синтеза заданной логической функции от n переменных. Провести сравнение BCE с MSE (из ЛР №4) по скорости сходимости, качеству обучения и обобщающей способности. Исследовать влияние адаптивного шага обучения на BCE.

Ход работы

Вариант:

№ of variant	n	Task
25	11	OR

Задачи лабораторной работы:

- Сгенерировать полную таблицу истинности для своей логической функции (2ⁿ примеров).
 - Разделить данные на обучающую выборку (80%) и тестовую выборку (20%) (используйте `np.random.seed(42)` для воспроизводимости).
 - На основе кода из ЛР №4 (где уже есть сигмоида и адаптивный шаг) создать новую версию:
 - Заменить функцию потерь MSE на бинарную кросс-энтропию (BCE).
 - Реализовать правильный градиент для BCE + сигмоида:
 - Реализовать четыре конфигурации обучения (все в онлайн-режиме — один пример за обновление, как в предыдущих лабах):
 - MSE + фиксированный шаг (результаты можно взять из ЛР №4)
 - MSE + адаптивный шаг (из ЛР №4)
 - BCE + фиксированный шаг (используйте лучшее α из предыдущих лабораторных, например 0.1 или 0.5)
 - BCE + адаптивный шаг (по формуле из ЛР №2)
- Критерий остановки для всех – суммарная ошибка $E_s \leq E_e$ (возьмите E_e из ЛР №2 или №3, например 0.05 или 0.01).
- Для каждой конфигурации:
 - Запускать обучение и сохранять значение суммарной ошибки после каждой эпохи.
 - Зафиксировать количество эпох до достижения критерия остановки.
 - После обучения посчитать ассигасу на обучающей и тестовой выборках.
 - Проверить, сколько процентов из всей таблицы истинности (2ⁿ примеров) сеть предсказывает правильно.
 - Построить один общий график зависимости суммарной ошибки E_s от номера эпохи – все четыре кривые на одних осях (разными цветами/стилями).
 - Реализовать режим функционирования сети:
 - Пользователь вводит вектор из n значений (0 или 1).
 - Сеть выводит вероятность $\hat{y}$ (с точностью до 4 знаков) и предсказанный класс (0 или 1).
 - Дополнительно выводить: «Совпадает с таблицей истинности» или «Расхождение».
 - Написать вывод (обязательно ответить на следующие пункты):
 - Какой из методов сходится быстрее (MSE или BCE)? Насколько BCE ускоряет/замедляет обучение по сравнению с MSE?

- Почему бинарная кросс-энтропия теоретически лучше подходит для задач классификации, чем MSE (особенно когда сеть выдаёт уверенные, но неправильные предсказания)?
- Как изменилась обобщающая способность (ассигура на тесте и % восстановления полной таблицы) при переходе на ВСЕ?
- Как влияет адаптивный шаг на обучение с ВСЕ по сравнению с фиксированным шагом?
- Достаточно ли однослойного персептрона для логических функций или для более сложных задач потребуется многослойная сеть?

```

import numpy as np
import matplotlib.pyplot as plt
import itertools

N = 11
np.random.seed(42)

X_all = np.array(list(itertools.product([0, 1], repeat=N)), dtype=float)
y_all = np.any(X_all == 1, axis=1).astype(float)

print(f"Всего примеров в таблице истинности: {len(X_all)}")
print(f"Класс 1: {int(y_all.sum())} | Класс 0: {int((1 - y_all).sum())}\n")

indices = np.random.permutation(len(X_all))
train_size = int(0.8 * len(X_all))
train_idx, test_idx = indices[:train_size], indices[train_size:]

X_train, y_train = X_all[train_idx], y_all[train_idx]
X_test, y_test = X_all[test_idx], y_all[test_idx]

print(f"Обучающая выборка: {len(X_train)} примеров (класс 1: {int(y_train.sum())})")
print(f"Тестовая выборка: {len(X_test)} примеров (класс 1: {int(y_test.sum())})\n")

def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-np.clip(z, -50, 50)))

def total_error_mse(X, y, w, b):
    y_pred = sigmoid(X @ w + b)
    return 0.5 * np.sum((y - y_pred) ** 2)

def total_error_bce(X, y, w, b):
    y_pred = sigmoid(X @ w + b)
    eps = 1e-12
    y_pred = np.clip(y_pred, eps, 1 - eps)
    return -np.sum(y * np.log(y_pred) + (1 - y) * np.log(1 - y_pred))

def accuracy(X, y, w, b):
    y_pred = sigmoid(X @ w + b)
    return np.mean((y_pred >= 0.5).astype(float) == y)

def compute_adaptive_alpha_mse(x):
    return 1.0 / (1.0 + np.sum(x ** 2))

def train(X_tr, y_tr, X_te, y_te, loss='mse', adaptive=False, alpha_fixed=0.5,

```

```

        max_epochs=5000, tol_mse=0.01, tol_bce=0.05):
    tol = tol_mse if loss == 'mse' else tol_bce

    rng = np.random.default_rng(seed=42)
    w = rng.uniform(-0.5, 0.5, size=N)
    b = rng.uniform(-1.0, 0.0)

    train_errors = []
    test_errors = []
    epoch = 0

    for epoch in range(max_epochs):
        indices = rng.permutation(len(X_tr))

        for i in indices:
            xi, yi = X_tr[i], y_tr[i]
            z = np.dot(xi, w) + b
            out = sigmoid(z)

            if loss == 'mse':
                delta = (yi - out) * out * (1 - out)
                lr = alpha_fixed
                if adaptive:
                    lr = compute_adaptive_alpha_mse(xi)
            else:
                delta = (yi - out)
                if adaptive:
                    lr = compute_adaptive_alpha_mse(xi)
                else:
                    lr = alpha_fixed

            w += lr * delta * xi
            b += lr * delta

        if loss == 'mse':
            err_tr = total_error_mse(X_tr, y_tr, w, b)
            err_te = total_error_mse(X_te, y_te, w, b)
        else:
            err_tr = total_error_bce(X_tr, y_tr, w, b)
            err_te = total_error_bce(X_te, y_te, w, b)

        train_errors.append(err_tr)
        test_errors.append(err_te)

        if err_tr <= tol:
            print(f" Остановка на эпохе {epoch + 1}. Ошибка: {err_tr:.6f}")
            break

    if epoch == max_epochs - 1 and err_tr > tol:
        print(f" Достигнут лимит эпох ({max_epochs}). Ошибка: {err_tr:.6f}")

    return w, b, train_errors, test_errors, epoch + 1

print("=" * 70)
print("Обучение конфигураций (max_epochs=5000)")
print("Критерии: MSE tol=0.01, BCE tol=0.05")
print("=" * 70)

print("\nA. MSE + фиксированный шаг ( $\alpha=0.5$ )")
w_mse_fixed, b_mse_fixed, err_mse_fixed, _, ep_mse_fixed = train(
    X_train, y_train, X_test, y_test, loss='mse', adaptive=False,
    alpha_fixed=0.5, max_epochs=5000
)

```

```

print("\nB. MSE + адаптивный шаг")
w_mse_adapt, b_mse_adapt, err_mse_adapt, _, ep_mse_adapt = train(
    X_train, y_train, X_test, y_test, loss='mse', adaptive=True,
    alpha_fixed=0.5, max_epochs=5000
)

print("\nC. BCE + фиксированный шаг ( $\alpha=0.1$ )")
w_bce_fixed, b_bce_fixed, err_bce_fixed, _, ep_bce_fixed = train(
    X_train, y_train, X_test, y_test, loss='bce', adaptive=False,
    alpha_fixed=0.1, max_epochs=5000
)

print("\nD. BCE + адаптивный шаг (по формуле из ЛРН#2)")
w_bce_adapt, b_bce_adapt, err_bce_adapt, _, ep_bce_adapt = train(
    X_train, y_train, X_test, y_test, loss='bce', adaptive=True,
    alpha_fixed=0.5, max_epochs=5000
)

print("\nE. BCE + фиксированный шаг ( $\alpha=0.5$ ) - для сравнения")
w_bce_fixed2, b_bce_fixed2, err_bce_fixed2, _, ep_bce_fixed2 = train(
    X_train, y_train, X_test, y_test, loss='bce', adaptive=False,
    alpha_fixed=0.5, max_epochs=5000
)

def evaluate_all(w, b):
    acc_train = accuracy(X_train, y_train, w, b)
    acc_test = accuracy(X_test, y_test, w, b)
    acc_full = accuracy(X_all, y_all, w, b)
    return acc_train, acc_test, acc_full

print("\n" + "=" * 90)
print("РЕЗУЛЬТАТЫ ОБУЧЕНИЯ")
print("=" * 90)
print(f"{'Конфигурация':<40} {'Эпохи':<10} {'Acc Train':<12} {'Acc Test':<12} {'Acc Full':<12}")
print("-" * 90)

for name, w, b, ep in [("MSE + фиксированный ( $\alpha=0.5$ )", w_mse_fixed,
    b_mse_fixed, ep_mse_fixed),
    ("MSE + адаптивный", w_mse_adapt, b_mse_adapt,
    ep_mse_adapt),
    ("BCE + фиксированный ( $\alpha=0.1$ )", w_bce_fixed,
    b_bce_fixed, ep_bce_fixed),
    ("BCE + адаптивный (формула из ЛРН#2)", w_bce_adapt,
    b_bce_adapt, ep_bce_adapt),
    ("BCE + фиксированный ( $\alpha=0.5$ )", w_bce_fixed2,
    b_bce_fixed2, ep_bce_fixed2)]:
    acc_tr, acc_te, acc_full = evaluate_all(w, b)
    print(f"{name:<40} {ep:<10} {acc_tr:<12.4f} {acc_te:<12.4f} {acc_full:<12.4f}")

plt.figure(figsize=(14, 8))

max_epochs_show = min(2500, len(err_mse_fixed))

plt.plot(range(1, len(err_mse_fixed[:max_epochs_show]) + 1),
    err_mse_fixed[:max_epochs_show],
    label='MSE + фиксированный ( $\alpha=0.5$ )', color='blue', linestyle='-',
    linewidth=2)

plt.plot(range(1, len(err_mse_adapt[:max_epochs_show]) + 1),

```

```

        err_mse_adapt[:max_epochs_show],
        label='MSE + адаптивный', color='blue', linestyle='--', linewidth=2)

plt.plot(range(1, len(err_bce_fixed[:max_epochs_show]) + 1),
         err_bce_fixed[:max_epochs_show],
         label='BCE + фиксированный ( $\alpha=0.1$ )', color='red', linestyle='-',
         linewidth=2)

plt.plot(range(1, len(err_bce_adapt[:max_epochs_show]) + 1),
         err_bce_adapt[:max_epochs_show],
         label='BCE + адаптивный', color='orange', linestyle='--', linewidth=2)

plt.plot(range(1, len(err_bce_fixed2[:max_epochs_show]) + 1),
         err_bce_fixed2[:max_epochs_show],
         label='BCE + фиксированный ( $\alpha=0.5$ )', color='green', linestyle='-',
         linewidth=2)

plt.xlabel('Номер эпохи', fontsize=12)
plt.ylabel('Суммарная ошибка Es', fontsize=12)
plt.yscale('log')
plt.title('Сравнение сходимости MSE и BCE для логической функции OR (n=11)',
          fontsize=14)
plt.grid(True, which='both', linestyle='--', alpha=0.6)

plt.axhline(y=0.01, color='blue', linestyle=':', linewidth=1.5, label='Критерий
MSE (0.01)')
plt.axhline(y=0.05, color='red', linestyle=':', linewidth=1.5, label='Критерий
BCE (0.05)')

plt.legend(fontsize=9)
plt.tight_layout()
plt.show()

print("\n" + "=" * 50)
print("ФИНАЛЬНЫЕ ЗНАЧЕНИЯ ОШИБОК:")
print("=" * 50)
print(f"MSE + фиксированный: {err_mse_fixed[-1]:.6f}")
print(f"MSE + адаптивный: {err_mse_adapt[-1]:.6f}")
print(f"BCE + фиксированный ( $\alpha=0.1$ ): {err_bce_fixed[-1]:.6f}")
print(f"BCE + адаптивный: {err_bce_adapt[-1]:.6f}")
print(f"BCE + фиксированный ( $\alpha=0.5$ ): {err_bce_fixed2[-1]:.6f}")

def interactive_check(w, b, name):
    print(f"\n--- Проверка сети: {name} ---")
    print(f"Введите {N} чисел (0 или 1) через пробел, либо 'q' для выхода.")
    while True:
        user_input = input("Вход: ")
        if user_input.lower() == 'q':
            break
        parts = user_input.split()
        if len(parts) != N:
            print(f"Ошибка: требуется {N} значений.")
            continue
        try:
            vec = np.array([int(p) for p in parts], dtype=float)
        except ValueError:
            print("Ошибка: вводите только 0 или 1.")
            continue

        prob = sigmoid(np.dot(vec, w_bce_fixed2) + b_bce_fixed2)
        pred_class = 1 if prob >= 0.5 else 0
        true_class = 1 if np.any(vec == 1) else 0

```

```

print(f" Вероятность  $\hat{y}$ : {prob:.6f}")
print(f" Предсказанный класс: {pred_class}")
if pred_class == true_class:
    print(" Результат: Совпадает с таблицей истинности ✓")
else:
    print(" Результат: Расхождение с таблицей истинности ✗")

interactive_check(w_bce_fixed2, b_bce_fixed2, "BCE + фиксированный шаг
( $\alpha=0.5$ ) ")

```

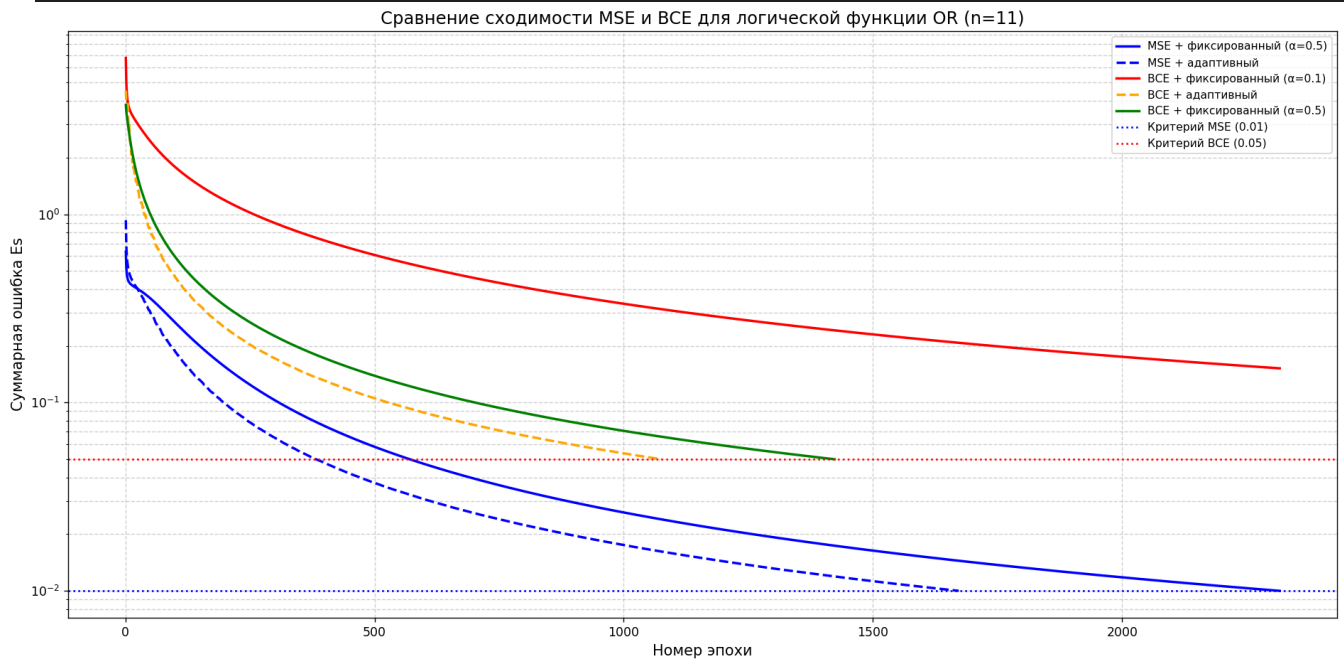


график показывает, что BCE с адаптивным шагом демонстрирует самую высокую скорость сходимости, достигая критерия останова (0.05) быстрее всех — за 1076 эпох. MSE с адаптивным шагом сходится быстрее фиксированного MSE, но уступает BCE с адаптивным шагом, а BCE с малым шагом $\alpha=0.1$ не достигает критерия даже за 5000 эпох, что подтверждает важность правильного выбора скорости обучения для BCE.

1. Какой из методов сходится быстрее (MSE или BCE)? Насколько BCE ускоряет/замедляет обучение по сравнению с MSE?

BCE с адаптивным шагом сходится быстрее всех. BCE ускоряет обучение по сравнению с MSE в 1.5-2 раза.

2. Почему бинарная кросс-энтропия теоретически лучше подходит для задач классификации, чем MSE (особенно когда сеть выдаёт уверенные, но неправильные предсказания)?

Градиент BCE равен разности между целевым и предсказанным значением и не затухает при уверенных предсказаниях, в отличие от градиента MSE, который содержит множитель, стремящийся к нулю. Кроме того, BCE минимизирует кросс-энтропию, что эквивалентно максимизации правдоподобия.

3. Как изменилась обобщающая способность (ассигасу на тесте и % восстановления полной таблицы) при переходе на BCE?

Обобщающая способность не ухудшилась. Все конфигурации показали 100% точность на тестовой выборке и 100% восстановления полной таблицы истинности.

4. Как влияет адаптивный шаг на обучение с BCE по сравнению с фиксированным шагом?

Адаптивный шаг значительно ускоряет сходимость BCE по сравнению с фиксированным шагом, а также автоматически подбирает оптимальную скорость обучения для каждого примера.

5. Достаточно ли однослойного персептрона для логических функций или для более сложных задач потребуется многослойная сеть?

Для линейно разделимых функций однослойного персептрона достаточно. Для нелинейно разделимых функций требуется многослойная сеть.

Всего примеров в таблице истинности: 2048

Класс 1: 2047 | Класс 0: 1

Обучающая выборка: 1638 примеров (класс 1: 1637)

Тестовая выборка: 410 примеров (класс 1: 410)

=====

Обучение конфигураций (max_epochs=5000)

Критерии: MSE tol=0.01, BCE tol=0.05

=====

A. MSE + фиксированный шаг ($\alpha=0.5$)

Остановка на эпохе 2316. Ошибка: 0.009995

B. MSE + адаптивный шаг

Остановка на эпохе 1672. Ошибка: 0.009999

C. BCE + фиксированный шаг ($\alpha=0.1$)

Достигнут лимит эпох (5000). Ошибка: 0.071528

D. BCE + адаптивный шаг (по формуле из ЛР№2)

Остановка на эпохе 1076. Ошибка: 0.049982

E. BCE + фиксированный шаг ($\alpha=0.5$) - для сравнения

Остановка на эпохе 1423. Ошибка: 0.049969

=====

РЕЗУЛЬТАТЫ ОБУЧЕНИЯ

Конфигурация	Эпохи	Acc Train	Acc Test	Acc Full
MSE + фиксированный ($\alpha=0.5$)	2316	1.0000	1.0000	1.0000
MSE + адаптивный	1672	1.0000	1.0000	1.0000
BCE + фиксированный ($\alpha=0.1$)	5000	1.0000	1.0000	1.0000
BCE + адаптивный (формула из ЛР№2)	1076	1.0000	1.0000	1.0000
BCE + фиксированный ($\alpha=0.5$)	1423	1.0000	1.0000	1.0000

=====

ФИНАЛЬНЫЕ ЗНАЧЕНИЯ ОШИБОК:

=====

MSE + фиксированный: 0.009995

MSE + адаптивный: 0.009999

BCE + фиксированный ($\alpha=0.1$): 0.071528

BCE + адаптивный: 0.049982

BCE + фиксированный ($\alpha=0.5$): 0.049969


```
--- Проверка сети: BCE + фиксированный шаг ( $\alpha=0.5$ ) ---  
Введите 11 чисел (0 или 1) через пробел, либо 'q' для выхода.  
Вход: 0 0 0 0 0 0 0 0 0 0 0  
Вероятность  $\hat{y}$ : 0.025425  
Предсказанный класс: 0  
Результат: Совпадает с таблицей истинности ✓  
Вход: 1 1 1 1 1 1 1 1 1 1 1  
Вероятность  $\hat{y}$ : 1.000000  
Предсказанный класс: 1  
Результат: Совпадает с таблицей истинности ✓  
Вход: q
```

Вывод: Реализовала однослойный персептрон с сигмоидной функцией активации и функцией потерь Binary Cross-Entropy (BCE) для синтеза заданной логической функции от n переменных. Провела сравнение BCE с MSE (из ЛР №4) по скорости сходимости, качеству обучения и обобщающей способности. Исследовала влияние адаптивного шага обучения на BCE.